

Loss Function in Sequence-to-Sequence Architectures

In sequence-to-sequence (seq2seq) models, such as Transformers, LSTMs, or GRU-based encoder-decoder models, the loss function plays a crucial role in guiding the model to generate accurate and fluent sequences. The objective is to maximize the likelihood of generating the correct output sequence given an input sequence. This discussion covers the formulation, interpretation, and practical considerations of the loss function used in training seq2seq models.

Understanding the Sequence-to-Sequence Setting

Let us define the input and output formally:

- **Input sequence:** $X = (x_1, x_2, \dots, x_m)$ (e.g., a sentence in English)
- **Target output sequence:** $Y = (y_1, y_2, \dots, y_n)$ (e.g., the corresponding translation in French)
- **Predicted output sequence:** $\hat{Y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$

The model is trained to generate Y one token at a time, predicting each token y_t given the input sequence and all previously generated tokens.

Defining the Loss Function

The probability of the correct target sequence given the input sequence is modeled as:

$$P(Y|X) = P(y_1, y_2, \dots, y_n|X) = \prod_{t=1}^n P(y_t|y_1, \dots, y_{t-1}, X),$$

where $P(y_t|y_{<t}, X)$ is the probability of generating the token y_t at time step t , conditioned on the input sequence X and previously generated tokens.

To train the model, we maximize this probability over the dataset, which is equivalent to minimizing the negative log-likelihood loss (NLL):

$$\mathcal{L} = - \sum_{t=1}^n \log P(y_t|y_1, \dots, y_{t-1}, X)$$

Why Negative Log-Likelihood (NLL)?

- Maximizing probability is the goal, but taking the negative log turns it into a minimization problem (which aligns with gradient-based optimization).
- Log probabilities help stabilize training by preventing numerical underflow when multiplying many small probabilities.

Extending the Loss to the Entire Dataset

For a dataset containing N training examples $\{(X^{(i)}, Y^{(i)})\}_{i=1}^N$, the total loss is given by:

$$\mathcal{L}_{\text{dataset}} = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{n_i} \log P(y_t^{(i)} | y_{<t}^{(i)}, X^{(i)})$$

where:

- n_i is the length of the target sequence for the i -th example.
- $P(y_t^{(i)} | y_{<t}^{(i)}, X^{(i)})$ is the probability of the correct token at step t , given by the model.

This ensures that the loss is averaged across all sequences in the dataset, making it independent of dataset size.

General Definition of $P(y_t | y_{<t}, X)$

In Natural Language Processing (NLP), the probability of a word $P(y_t | y_{<t}, X)$ is a fundamental concept used across multiple tasks. This probability models the likelihood of a word y_t appearing given the prior context $y_{<t}$ and, in some cases, an external input X (such as a source sentence in machine translation). Different NLP tasks define and compute this probability differently.

This note provides a comprehensive overview of how $P(y_t | y_{<t}, X)$ is defined across major NLP tasks and models, along with the detailed mathematics behind different architectures.

The probability of the next token given previous tokens and optional input X is computed as:

$$P(y_t | y_{<t}, X) = f(y_{<t}, X; \theta)$$

where:

- $f(\cdot)$ is a function parameterized by a neural network (e.g., RNN, Transformer, Softmax classifier).
- $y_{<t}$ represents the sequence of previous tokens.
- X represents optional input (e.g., a source sentence in translation, context in dialogue systems).
- θ are the learnable parameters of the model.

The computation of this probability varies depending on the specific NLP task and model architecture.

Probability Computation Across NLP Architectures

N-Gram Models

N-gram models estimate $P(y_t | y_{<t})$ using fixed-window approximations:

$$P(y_t | y_{<t}) \approx P(y_t | y_{t-N+1}, \dots, y_{t-1}).$$

Using frequency counts:

$$P(y_t | y_{t-N+1}, \dots, y_{t-1}) = \frac{\text{Count}(y_{t-N+1}, \dots, y_t)}{\text{Count}(y_{t-N+1}, \dots, y_{t-1})}$$

Efficient but lacks long-range dependencies.

RNN-Based Language Models

RNNs model sequential dependencies by maintaining a hidden state:

$$h_t = f(W_h h_{t-1} + W_y y_{t-1})$$

where:

- h_t is the hidden state at time t .
- W_h and W_y are learned weight matrices.

The probability of the next token is:

$$P(y_t | y_{<t}) = \text{softmax}(W_o h_t)$$

Pros: Captures long-range dependencies but struggles with vanishing gradients.

LSTM and GRU-Based Models

LSTMs and GRUs improve RNNs by introducing memory gates:

$$h_t = \text{LSTM}(h_{t-1}, y_{t-1})$$

The probability computation remains:

$$P(y_t | y_{<t}) = \text{softmax}(W_o h_t)$$

Pros: Handles long dependencies better than RNNs.

Transformer-Based Models

Transformers compute h_t differently, using **self-attention** and feedforward layers.

Self-Attention Mechanism

The hidden state is computed via self-attention:

$$h_t = \sum_{i=1}^{t-1} \alpha_{ti} V_i$$

where attention weights are computed as:

$$\alpha_{ti} = \frac{\exp((Q_t \cdot K_i) / \sqrt{d_k})}{\sum_j \exp((Q_t \cdot K_j) / \sqrt{d_k})}$$

where Q, K, V are query, key, and value matrices.

Output Projection W_o in Transformers

The final word probability is computed as:

$$z_t = W_o h_t$$

$$P(y_t | y_{<t}) = \frac{\exp(z_t^{(y_t)})}{\sum_{y' \in V} \exp(z_t^{(y')})}$$

Pros: Handles long-range dependencies efficiently.

Loss Function for Sequence Generating Models

Training models involves minimizing the **Negative Log-Likelihood (NLL)**:

$$\mathcal{L} = - \sum_{t=1}^n \log P(y_t | y_{<t}, X)$$

For Transformers and RNNs, this translates to **cross-entropy loss**:

$$\mathcal{L} = - \sum_{t=1}^n y_t \log \hat{y}_t$$

where:

- y_t is the true token (one-hot encoded).
- $\hat{y}_t$ is the predicted probability.

Used in: Language Modeling, Machine Translation, Text Generation.

Summary Table of $P(y_t | y_{<t}, X)$ Computation

Model	Computation of $P(y_t y_{<t}, X)$	Key Equation
N-Gram	Uses fixed-size context	$P(y_t y_{t-N+1}, \dots, y_{t-1})$
RNN	Hidden state recurrence	$P(y_t y_{<t}) = \text{softmax}(W_o h_t)$
LSTM/GRU	Memory cell updates	$P(y_t y_{<t}) = \text{softmax}(W_o h_t)$
Transformer	Self-attention	$P(y_t y_{<t}) = \frac{\exp(W_o h_t)}{\sum_{y' \in V} \exp(W_o h_t)}$

Table 1: Computation of $P(y_t | y_{<t}, X)$ across different NLP architectures

Training with Teacher Forcing

During training, the model uses **teacher forcing**, meaning:

- Instead of using its own generated tokens at previous steps, it is fed the **ground truth tokens** $y_{1:t-1}$ to predict y_t .
- This stabilizes training by preventing the model from accumulating errors early in the sequence.

However, during **inference**, the model does not have access to ground-truth tokens and must generate sequences autonomously, token by token.

Handling Special Cases

Padding in Variable-Length Sequences

Since training samples have different sequence lengths, shorter sequences are padded to match the longest sequence in a batch. **Padding tokens must be ignored in the loss computation**, so we introduce a masking term:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{n_i} M_t^{(i)} \log P(y_t^{(i)} | y_{<t}^{(i)}, X^{(i)})$$

where:

- $M_t^{(i)}$ is **1** if $y_t^{(i)}$ is a real token, **0** if it is a padding token.

This ensures that the model is not penalized for predicting padding tokens.

Label Smoothing

Instead of using a **one-hot vector** for each token, we can smooth the target distribution:

$$y_t^{\text{smoothed}} = (1 - \alpha)y_t^{\text{one-hot}} + \frac{\alpha}{|V|}$$

where α is a small constant (e.g., 0.1). This **prevents overconfidence** and helps **generalization**.

Final Summary

- The loss function in seq2seq models is the **Negative Log-Likelihood (NLL)**, computed per token, per sequence, and averaged over the dataset.
- The decoder generates one token at a time, predicting y_t given previous tokens.
- Softmax is used to compute probabilities over the vocabulary.
- Teacher forcing is used during training, but inference requires the model to generate tokens individually.
- Special cases like padding and label smoothing are handled to improve training stability.

1 Input–Output Structure and Loss in Sequence-to-Sequence Models

Let $\mathcal{V} = \{w_1, w_2, \dots, w_D\}$ denote the vocabulary, where $D = |\mathcal{V}|$ is the total number of tokens (e.g., words). Each token is represented as a one-hot vector:

$$x_t \in \{0, 1\}^D, \quad (x_t)_i = \begin{cases} 1, & \text{if } w_i \text{ is the } t\text{-th token,} \\ 0, & \text{otherwise.} \end{cases}$$

Thus, conceptually, the model’s input and output spaces are both D -dimensional.

1.1 Embedding and Encoder Representation

In practice, models do not operate directly on one-hot vectors. Instead, an *embedding matrix*

$$E \in \mathbb{R}^{D \times d}$$

maps each one-hot vector into a dense embedding space of dimension $d \ll D$:

$$h_t^{(0)} = E^\top x_t.$$

For an input sentence $X = (x_1, \dots, x_n)$, the encoder produces contextual hidden states

$$H = \text{Encoder}(h_1^{(0)}, \dots, h_n^{(0)}) = (h_1, h_2, \dots, h_n),$$

which summarize semantic and syntactic information about the source sequence.

1.2 Decoder and Conditional Probability Factorization

The decoder generates the target sentence $Y = (y_1, \dots, y_m)$ one token at a time, conditioning on both the encoded representation H and previously generated tokens $y_{<t}$. The probability of the entire sequence is factorized as

$$P(Y | X) = \prod_{t=1}^m P(y_t | y_{<t}, X).$$

At each time step t , the decoder produces a hidden state s_t :

$$s_t = \text{Decoder}(y_{<t}, H),$$

followed by a linear projection to vocabulary logits:

$$z_t = W_o s_t + b, \quad W_o \in \mathbb{R}^{D \times d}.$$

The softmax layer then converts logits into a probability distribution over the vocabulary:

$$p_t = \text{softmax}(z_t), \quad p_{t,i} = \frac{e^{z_{t,i}}}{\sum_{j=1}^D e^{z_{t,j}}}.$$

Hence, the output layer has D nodes, corresponding to all possible tokens in $\mathcal{V}$.

1.3 Training Objective

Let the ground-truth target token at time t be represented by a one-hot vector $y_t^{(\text{onehot})} \in \{0, 1\}^D$. The cross-entropy loss at each decoding step is

$$\ell_t = - \sum_{i=1}^D y_{t,i}^{(\text{onehot})} \log p_{t,i} = - \log p_t(y_t),$$

and the total sequence loss is

$$\mathcal{L}(Y | X) = - \sum_{t=1}^m \log P(y_t | y_{<t}, X).$$

Minimizing this loss trains the model to assign high probability to the correct next token given the source sequence and the previously generated tokens.

1.4 Summary

Conceptually, both the input and output layers are D -dimensional, corresponding to the vocabulary size. The model learns a mapping

$$E^\top : \{0, 1\}^D \rightarrow \mathbb{R}^d \quad \text{and} \quad W_o : \mathbb{R}^d \rightarrow \mathbb{R}^D,$$

such that the conditional probabilities $P(y_t | y_{<t}, X)$ approximate the true data distribution. The entire training process aligns the one-hot target vectors with the model's predicted probability distributions over $\mathcal{V}$.